

Revon: A Content-Addressed Versioned Data Store: A Git-Inspired Approach to Structured Data Versioning Using Merkle Hash Tries

Atharva Sheersh Pandey¹, Adhyan Jain¹, Poornima Nedunchezian²
atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in;
poornima.n@vit.ac.in

School of Computer Science and Engineering
VIT Vellore

Abstract

Versioning structured datasets requires durable history, efficient sparse updates, historical reconstruction, and comparison without copying every record. Revon is a Python prototype built around an immutable fixed-depth Merkle hash trie; Revon-H adds addressed changesets and adaptively selects log aggregation or hash-pruned differencing. We evaluate Snapshot, Log-only, forced-Merkle Revon-M, Revon-H, and Dolt 2.3.1 on deterministic workloads up to 100,000 rows. The primary run contains 540 executions. Across eight workloads, median paired Dolt/Revon-H diff ratios range from 7.14 to 625.10; all seven-trial 95% bootstrap intervals favor Revon-H latency in this tested workflow. A separate three-seed study across payload sizes, history lengths, and update patterns finds the 50-commit result inconclusive. These measurements compare the declared workflows on one Windows host and WSL2 on the same physical computer. They support a workload-dependent prototype design, not general or production-level superiority.

1 Introduction

Version control for structured data is more than archiving files. A useful system must retain exact versions, apply sparse mutations without rebuilding the complete state, reconstruct historical versions, and identify changes efficiently. Git provides the starting idea--immutable objects named by content and commits linked to their parents [1]--but organizes those objects around files and directories. A record store presents a different workload: keys remain stable, update density varies, and two versions may be separated by a handful of edits or by a long sequence of commits.

Revon evaluates whether a deliberately simple fixed-depth Merkle hash trie can unite incremental structural sharing with an operation index. Its primary variant, Revon-H, uses the log path for short or sparse intervals and switches to the Merkle path when the accumulated operations cross a calibration boundary fixed before evaluation. SQLite provides atomic durable storage, but does not determine the versioning algorithm.

Dolt is the production comparator because it combines SQL semantics, a Git-style commit graph, and content-addressed Proolly Trees [2], [3]. Revon is not presented as a feature replacement for Dolt. The comparison measures a smaller in-process Python prototype against a Dolt CLI workflow and does not isolate their tree algorithms.

1.1 Research Questions

- RQ1: What latency and storage trade-offs does Revon-H exhibit against Snapshot, Log-only, and Dolt across dataset size, mutation density, and locality?
- RQ2: Does adaptive log/Merkle selection improve diff latency relative to a forced-Merkle ablation without changing correctness?
- RQ3: How sensitive are fixed-trie latency, storage, and examined work to branching factor and depth?

1.2 Contributions

- A canonical SHA-256-addressed fixed-depth trie with incremental multi-key path rewriting.
- A hybrid diff policy combining addressed changesets with hash-pruned tree comparison.
- An atomic SQLite repository that persists objects, commits, aliases, metrics, and HEAD and verifies hashes after reopen.

- A five-system benchmark with shared deterministic workloads, common canonical diff and check-out outputs, two separately reported Dolt initial-import workflows, balanced randomized execution order, external process-tree RSS sampling, raw evidence, and an internal evidence-integrity audit.
- A controlled branching-factor/depth sensitivity study that prevents treating $b=8$, $d=4$ as universally optimal.

2 Literature Review and Related Work

2.1 Content Addressing and Persistent Structures

Merkle's authenticated tree construction established that a compact root digest can commit to a larger collection [4], while persistent-data-structure theory formalized path copying and bounded-overhead access to historical versions [5]. Git applies content-addressed blobs, trees, and commits to software history [1], and IPFS generalizes content-addressed links into a versionable Merkle DAG [6]. Revon adopts immutable object identity and parent links, but routes structured keys by hash into a persistent trie rather than mirroring a filesystem hierarchy.

Merkle Search Trees combine search-tree ordering with Merkle authentication for efficient state reconciliation [7]. Revon shares hash-pruned comparison but intentionally uses fixed-depth routing and a centralized commit history rather than a replicated CRDT. Content-defined chunking, established in LBFS for redundancy detection [8], also explains why history-independent boundaries are a credible storage improvement beyond the current fixed buckets.

2.2 Structured Dataset Versioning

Dataset-versioning research exposes a broader storage-retrieval design space. DataHub motivates collaborative dataset history at scale [9]. Decibel integrates branching into a relational storage engine and compares version-first, tuple-first, and hybrid layouts [10]. OrpheusDB bolts versioning onto a conventional DBMS and optimizes partitioning for version retrieval [11]. The dataset-versioning model in [12] formalizes the conflict between storing more materialized state and paying more reconstruction cost. Delta Lake represents the complementary log-and-checkpoint design for ACID tables and time travel [13]. These systems motivate reporting commit, checkout, diff, and storage together rather than selecting a single metric.

2.3 System Design Gap

Noms represents structured data as a Merkle DAG of immutable chunks and supports efficient diff and synchronization [14]. ForkBase combines content addressing, fork semantics, and duplicate-content detection for forkable applications [15]. Dolt stores table indexes as content-addressed Prolly Trees: ordered, B-tree-like structures with history-independent chunk boundaries, structural sharing, and native diff [2], [3]. Revon differs by using fixed-depth hash routing and by exposing an explicit alternative operation-log path. This simplifies instrumentation, but gives up Dolt's ordered range behavior, SQL surface, mature branching, merging, and production engineering.

Prior work	Core representation	Relationship to Revon
Persistent DS [5]	Path copying	Formal structural-sharing basis
Git [1]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [6]	Merkle DAG	General content-addressed versioning
MST [7]	Merkle search tree	Authenticated ordered reconciliation
DataHub [9]	Version graph	Collaborative dataset motivation
Decibel [10]	Relational layouts	Native database branching
OrpheusDB [11]	Partitioned versions	Storage/retrieval optimization
Delta Lake [13]	Log/checkpoints	ACID time travel at data-lake scale
Noms [14]	Merkle DAG	Structured content addressing
ForkBase [15]	Forkable CAS	Fork and deduplication semantics
DoltHub [2,3]	Prolly Trees	Production SOTA comparator

Table 1: Literature-review matrix and the gap addressed by Revon.

2.4 Design Implications and Research Positioning

Three separate costs recur in the literature: retained history, creation of a new version, and later reconstruction or comparison. A snapshot keeps reconstruction straightforward by copying unchanged records. A log avoids that copy at commit time, but the deferred work returns during replay, compaction, or checkpointing. Path-copying persistent structures take a third route: only the paths changed by an update are new, while the remaining topology is shared [5]. The storage-versus-recreation model in [12] makes this trade-off explicit. Decibel and OrpheusDB likewise show that one physical layout does not dominate scans, version retrieval, and storage for every branching workload [10], [11]. For that reason, our evaluation reports import, commit, checkout, diff, and storage rather than using one measurement as a proxy for the system.

Change detection is another point of separation. Comparing two complete states does not need a detailed history, although its work grows with the states being scanned. Replaying changesets instead makes the interval length decisive and requires an unbroken ancestry path with complete changesets. With Merkle differencing, matching subtree hashes stop the search; only mismatching branches are opened. The observed cost then depends on trie shape, leaf occupancy, key distribution, and where updates land. The MST retains key order [7]. Noms and Dolt use content-defined boundaries to form history-independent Prolly Trees [14], [3], while ForkBase reuses addressed content across objects, branches, and versions [15]. Revon chooses fixed hash-derived routes. This choice removes ordered range access and adaptive boundaries, but gives the experiment a stable geometry whose depth and branching factor can be measured.

The gap addressed here is not a new content-addressed store, but a controlled study in which two correct diff algorithms share one durable history and are chosen from observable properties of the requested interval. Every Revon-H commit records both a Merkle root and an addressed changeset, so changing the diff path does not change the meaning of a version. We expect log aggregation to suit short, sparse intervals and hash pruning to become preferable after many operations. We also expect b and d to change latency, examined work, and storage. Snapshot and Log-only provide representation baselines; Revon-M removes the selector; Revon-H retains it; and Dolt provides a production reference point. This design tests the hybrid mechanism while keeping the system-level Dolt comparison separate from any claim about intrinsic tree superiority.

3 Revon Design

Revon-H architecture

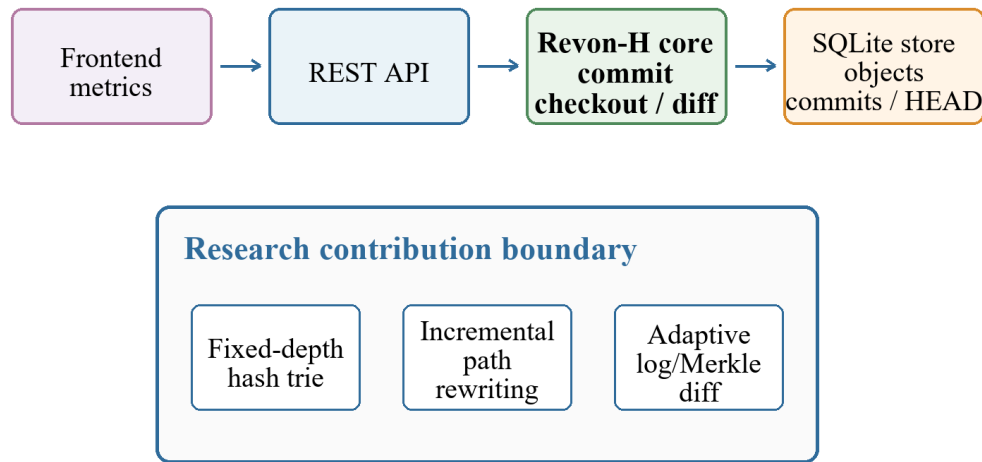


Figure 1: Revon system architecture and the boundary of its research contribution.

3.1 Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON, then names each object with its SHA-256 digest. The commit digest includes the root, parent, changeset, message, and timestamp. Labels such as v1 and v2 are only human-readable aliases; they do not determine object identity. The durable HEAD reference stores the current commit.

3.2 Fixed-Depth Merkle Hash Trie

For the main experiment we set $b=8$ and $d=4$. Each level reads three bits from SHA-256(key), so twelve routing bits address 4,096 possible leaf buckets. A bucket still stores the full key, which means that two keys sharing a routing prefix remain distinguishable. Under uniform hashes the expected bucket occupancy is N/b^d . We test other b,d pairs separately instead of assuming that this default is optimal.

3.3 Incremental Hashing and Commit

A mutation batch is canonicalized, routed, and grouped by shared prefixes. Each affected leaf is rewritten once, and only the union of affected ancestor paths is rebuilt. Unchanged child hashes are copied into new internal nodes, providing structural sharing. One SQLite transaction then inserts new objects, the changeset, commit metadata, version alias, measurements, and updated HEAD. Reopen verifies payload hashes and references.

Incremental commit and hybrid diff

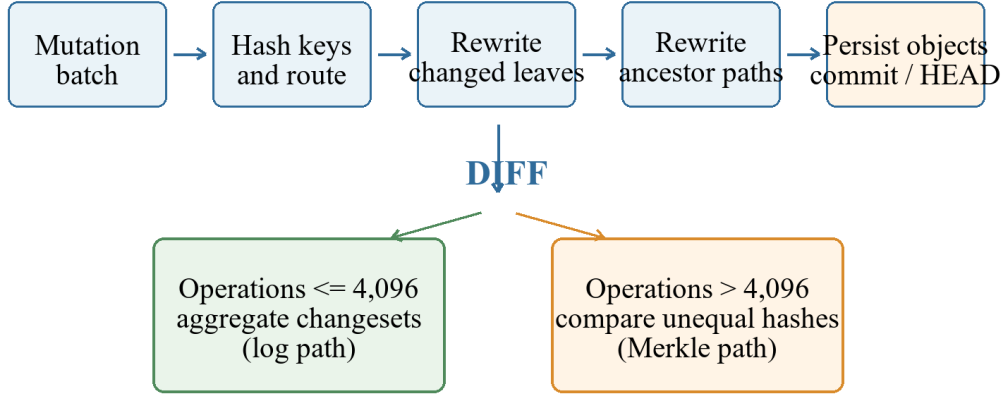


Figure 2: Incremental commit path and adaptive diff decision.

3.4 Hybrid Diff

The log path walks the ancestor chain and aggregates key operations. The Merkle path recursively compares roots, prunes equal subtree hashes, and examines full leaf entries only below unequal paths. Revon-H counts accumulated ancestor operations and uses the log at or below 4,096 operations; it uses Merkle above that boundary. Revon-M forces the Merkle path and therefore serves only as an ablation.

4 Experimental Methodology

4.1 Variants and Workloads

Five comparison systems receive identical logical states and mutation batches: a complete Snapshot per version, an initial state plus operation-log baseline, forced-Merkle Revon-M, adaptive Revon-H, and Dolt 2.3.1 using a keyed SQL table and native JSON diff. Dolt initial import is tested two ways: batched SQL INSERT and CSV loaded with `dolt table import -r`; CSV serialization is included in the latter workflow's initial-import time. The import variants share the same Dolt engine for later operations, and their import times are kept distinct.

Scenario	Rows	Comm.	Changes	Locality
Small sparse	1,000	10	10	spread
Medium sparse	10,000	10	10	spread
Medium dense	10,000	10	1,000	spread
Large sparse	100,000	10	100	spread
Application-key local	100,000	10	100	lexically adjacent keys
Range local	100,000	10	100	contiguous sorted-key intervals
Repeated key	100,000	10	100	repeated small key set
Hash-route local	100,000	10	100	shared 9-bit SHA-256 prefix

Table 2: Final evaluation workload matrix.

4.2 Measurement Contract

Initial import measures the first durable state; incremental commit is the within-trial median across ten commits; diff and checkout both materialize common outputs inside their timed regions. Diff is sorted changed keys with old/new SHA-256 value hashes encoded as canonical JSON. Checkout is the complete sorted key/value state encoded as canonical UTF-8 JSON. Dolt JSON parsing and nor-

malization are included. Correctness compares the full state and hash contract with the workload oracle. Dolt SQL-path commit timing includes SQL mutation, add, and commit; commit hashes are retained for historical checkout and no version tags are created. Timed operations run without tracemalloc. An external parent process samples process-tree RSS every 10 ms for each isolated primary trial, including Dolt children; sampling is outside timed operations. A separate Windows sensitivity telemetry run has 405 rows (315 measured) with RSS, CPU, read/write bytes, and serial commit throughput in every row; all rows passed correctness checks and the schema-2 audit. These counters include setup and correctness work, not only timed operations. The WSL2 replication has incomplete I/O coverage and used the same physical host. Work-examined units differ by system and are not normalized.

4.3 Repetition, Calibration, and Integrity

The primary matrix has two warm-ups and seven measured trials per configuration: 120 warm-ups and 420 measured runs across calibration and evaluation. The primary five-system evaluation contains 280 measured runs, including 56 SQL-path Dolt runs; 56 additional measured runs use Dolt bulk import. Model order is counterbalanced and recorded. Each system/trial runs in a clean worker. The primary run reuses one deterministic workload per scenario, regenerated and digest-checked by the internal audit. Run b7a4ec7b610742d9b827f43fd5092fe2 used Windows-11-10.0.26200-SP0 and Python 3.14.3. For each scenario and comparison, latency ratios are formed from matching trial numbers; a 20,000-resample percentile bootstrap of the seven paired ratios supplies a 95% interval. These intervals describe run-to-run variation conditional on the fixed workload and host, not variation across datasets or machines. Intervals that include parity are classified as inconclusive; a 5% practical margin is used as an analyst-selected label, not as an external standard.

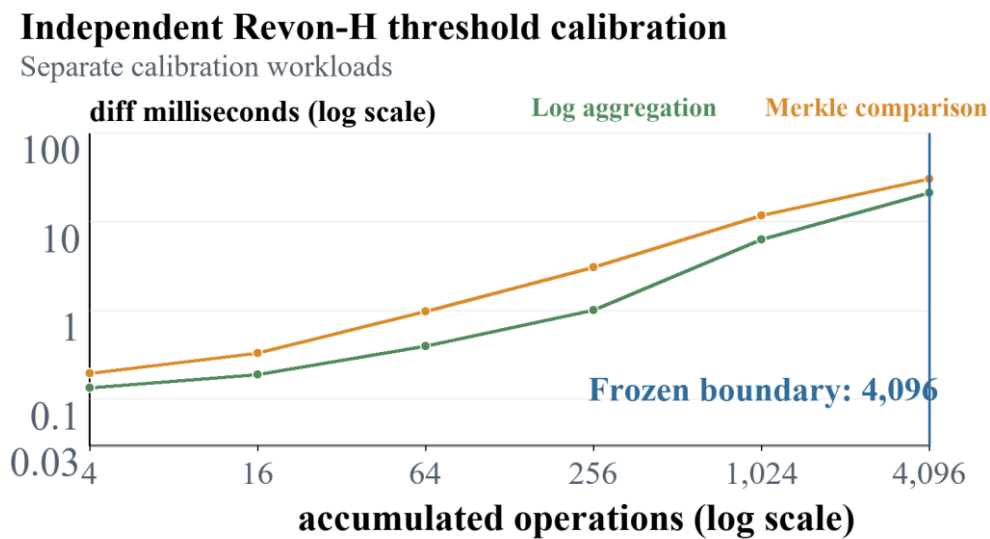


Figure 3: Independent threshold calibration; 4,096 is the largest sampled log-favorable point.

The 4,096-operation boundary is a frozen policy for this machine, not a universal crossover. All 540 primary executions completed successfully and passed state and diff checks. The audit retained all 127 Tukey outlier candidates; no observation was deleted. A separate sensitivity run varies seed, payload size, history length, and update pattern; it is kept separate from the primary tables.

5 Results

5.1 Final Diff Performance

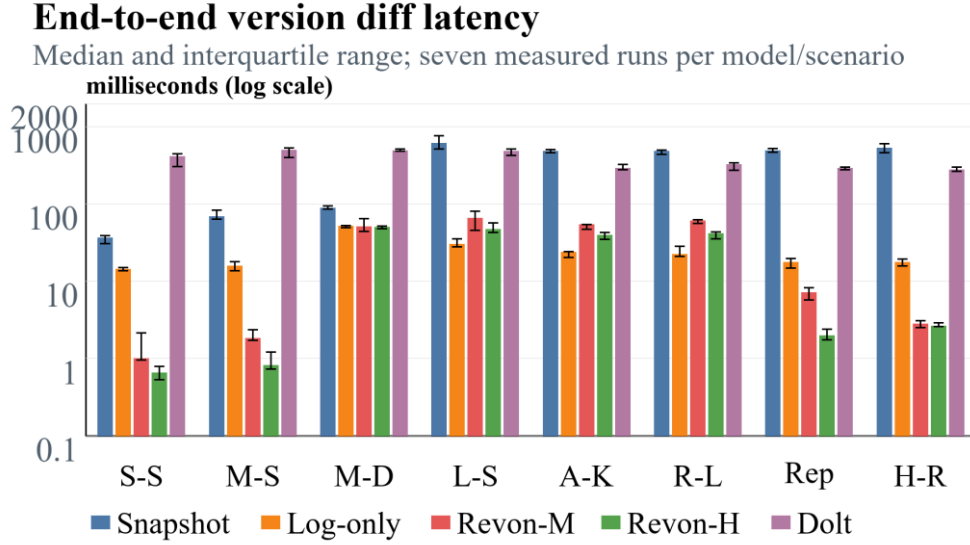


Figure 4: Diff latency across eight scenarios, including separate application-key, range, repeated-key, and hash-route update locality; lower is better.

Scenario	Path	H ms	Dolt ms	Dolt/H ratio [95% CI]
Small sparse	log	0.659	411.69	625.1 (537.1-658.7)
Medium sparse	log	0.814	503.31	482.5 (393.4-652.0)
Medium dense	merkle	50.391	488.42	10.3 (9.0-10.8)
Large sparse	log	46.834	483.08	10.8 (8.2-12.4)
Application-key local	log	39.361	289.87	7.1 (6.8-9.8)
Range local	log	41.359	325.04	7.8 (6.8-11.2)
Repeated key	log	1.992	293.72	141.6 (109.5-173.2)
Hash-route local	log	2.620	278.22	102.4 (97.5-104.0)

Table 3: Revon-H versus SQL-path Dolt diff latency; the final column reports the median paired trial ratio and percentile-bootstrap 95% interval.

Across the eight scenarios, every paired Dolt/Revon-H diff ratio is above one and its 95% interval excludes parity, indicating lower Revon-H latency for this tested workflow. Table 3 reports the median of the seven same-trial ratios and its bootstrap interval. This remains an in-process Revon API versus Dolt CLI workflow comparison: Dolt's CLI and SQL costs are included, so the ratios are not engine-only speedups. The lexical-key case is application-key local; range-local, repeated-key, and hash-route-local updates remain distinct workloads.

5.2 Hybrid Ablation

In the primary run, 3 of eight paired Revon-M/Revon-H diff intervals include parity and are inconclusive. A separate robustness matrix covers 10,000 rows, three seeds, 32- and 128-byte values, 10- and 50-commit histories, and spread, range-local, and repeated-key updates. Revon-M/Revon-H diff ratios above one mean Revon-M was slower. On Windows, baseline, 128-byte payload, range-local, and repeated-key ratios were 1.58 [1.23, 1.87], 1.62 [1.38, 1.72], 1.64 [1.38, 1.77], and 2.24 [2.02, 2.38]. The 50-commit interval crossed parity. WSL2 showed the same direction for those four factors and parity overlap for the 50-commit history. The paired ablation is more direct evidence about adaptive selection than cross-system ratios, but this three-seed, one-host sensitivity study does not establish general superiority.

5.3 Commit, Checkout, and Storage

Revon-H and Dolt operational latency

Median and interquartile range; Revon API vs Dolt CLI

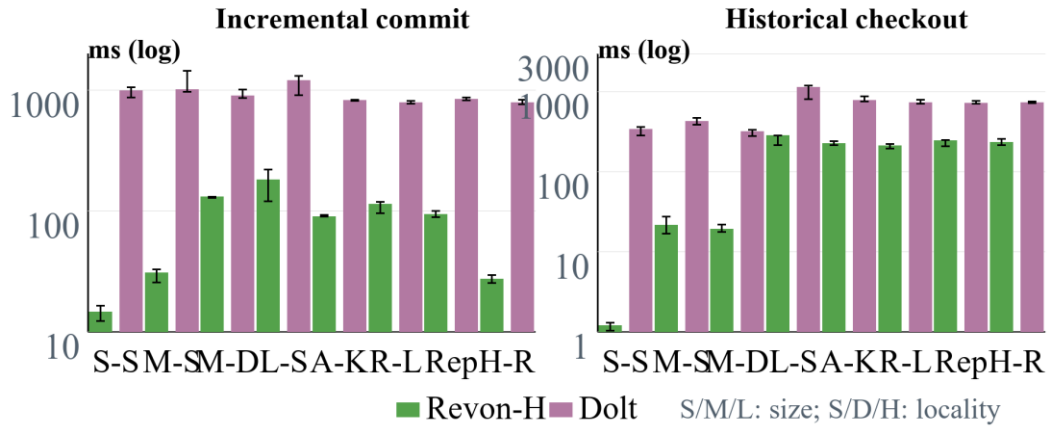


Figure 5: Revon-H and Dolt operational latency; lower is better.

Across all eight scenarios, paired Dolt/Revon-H intervals exclude parity for initial import, incremental commit, diff, and historical checkout. The full paired ratios and intervals are in `paired_ratio_uncertainty.csv`. Initial-import workflow medians across scenario medians were 1,814.83 ms for batched SQL INSERT, 1,701.25 ms for Dolt bulk import, and 1,131.24 ms for Revon-H. In the paired SQL/bulk-import analysis, four scenario intervals favor bulk import and four include parity; there is no general bulk-import advantage. These are workflow measurements; they do not establish engine-level or production-system superiority.

Repository directory size is a whole-workflow footprint, not normalized algorithm storage. Across the eight scenarios, Revon-H/Dolt repository-footprint ratios ranged from 1.99 to 6.90. Median Revon-H/Dolt ratios after dividing each directory by final UTF-8 logical key/value payload bytes were 3.82; the corresponding residual ratio was 6.82. The residual is not pure metadata: it includes serialization, indexes, metadata, and compression effects. Post-compaction mean footprints for one measured sample per scenario were Revon-M 22.08 MiB, Revon-H 22.08 MiB, Dolt SQL 2.17 MiB, and Dolt bulk import 2.19 MiB. Dolt was compacted offline with ``dolt gc`` and Revon SQLite repositories with ``VACUUM``; compaction time is outside latency and RSS. These are sampled outcomes, not a pure metadata comparison or universal steady-state result.

5.4 Trie-Parameter Sensitivity

Fixed-trie geometry sensitivity

10,000 rows; forced-Merkle diff; 10 warm-ups and 35 measured runs

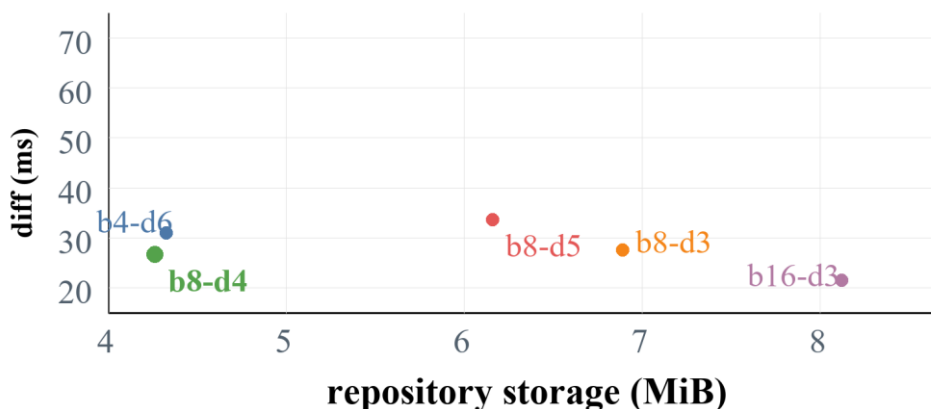


Figure 6: Fixed-trie sensitivity; dots show median diff latency, and the axis extends above the largest measured p75.

The corrected fixed-trie sensitivity run contains 45 successful executions: 10 warm-ups and 35 measured runs. b16-d3 had the lowest median diff (21.46 ms) but used 8.12 MiB; b8-d4 measured 26.74 ms and used the least storage (4.26 MiB). The over-deep b8-d5 design measured 33.68 ms for diff and used 44.5% more storage than b8-d4. Thus b8-d4 is a storage-conscious balanced default, not a universal optimum.

6 Discussion

RQ1 shows scenario-dependent workflow latency and storage trade-offs across the declared synthetic update patterns. A separate three-seed sensitivity study found consistent Revon-H diff advantages over the forced-Merkle ablation for four tested factors on Windows and WSL2. The 50-commit case remained inconclusive on both. Lexically adjacent application keys, sorted-range updates, repeated-key updates, and hash-route locality are different update patterns. None of these experiments measures ordered range queries, concurrent reads or writes, or production throughput.

RQ2 is supported more directly for the tested short and sparse histories: the paired Revon-M/Revon-H ablation shares the durable trie, while the added 50-commit case that crossed the frozen operation boundary showed no resolved difference. This does not establish a universal selector advantage.

RQ3 rejects a universal geometry. Shallow routing lowers internal traversal but creates larger leaf buckets and higher storage; deeper routing lowers occupancy but increases internal-node work. The suitable configuration therefore depends on which requirement dominates the workload: latency, storage, or worst-case leaf-bucket scanning.

The Dolt ratios need a system-level interpretation. Dolt performs SQL parsing, schema management, version-control operations, and production durability work that the Revon prototype does not. Dolt's Prolly Trees also serve ordered queries, branches, merges, and remote workflows. Our timings compare the declared interfaces used in the experiment. They do not isolate the tree algorithms and therefore cannot show that a fixed trie is inherently faster than a Prolly Tree.

7 Limitations and Threats to Validity

- Runtime and interface asymmetry: Revon executes in-process in Python, whereas Dolt is a mature Go executable invoked repeatedly through CLI and SQL interfaces.
- Feature asymmetry: Revon supports one writer and a linear history, but not SQL, branches, merges, remotes, schema evolution, or production concurrency.

- Synthetic scope: the primary study reaches 100,000 rows; the additional sensitivity study uses 10,000-row flat key-value tables, three seeds, 32- and 128-byte values, 10- and 50-commit histories, and three update patterns. It does not cover alternate schemas, public multi-table datasets, or one-million-row data.
- Environment scope: the primary run used one Windows 11 host. The sensitivity run also used WSL2 Ubuntu on that same physical computer, not an independent Linux machine. No external researcher or clean clone has independently reproduced the timings; the internal audit checks evidence consistency but does not reproduce wall-clock results.
- Calibration boundary: 4,096 is the largest sampled operation count that favored the log; this does not establish a universal crossover.
- Memory result: Across eight primary-evaluation scenarios, median sampled whole-worker process-tree RSS was collected for Snapshot, Log-only, Revon-M, Revon-H, and SQL-path Dolt. The values include common workload/oracle state and adapter processes, so they are not product-only memory. The parent sampled every 10 ms outside timed operations; peaks shorter than 10 ms may be missed, and the monitor can create small system-level scheduling pressure. Dolt bulk-import rows are not pooled with SQL-path Dolt for this comparison.
- Work-unit mismatch: keys, log operations, trie-node pairs, and Dolt's internal work do not form a single normalized measure.
- Uncertainty: all Tukey candidates were retained. The primary seven-trial ratio intervals are paired bootstrap intervals conditional on each fixed scenario. The additional seed-cluster bootstrap used only three seeds. Neither analysis establishes performance over a population of datasets or machines.

8 Future Work

A next version should choose its structure from observed workload statistics. Key count, prefix skew, mutation density, and leaf occupancy could guide the choice among measured trie geometries instead of fixing b and d for every repository. The diff selector could recalibrate periodically and consider history distance, changed-prefix density, and observed I/O cost alongside its present operation-count rule.

Storage efficiency could be improved with binary node records, separate value chunks, packed objects, compression, reachability checks, and garbage collection. An ordered Merkle index or content-defined chunk boundaries could add range scans and a history-independent layout while leaving the fixed trie as the controlled baseline.

To move beyond a linear prototype, Revon still needs named branches, merge commits, conflict reporting, schema-aware records, concurrent readers and writers, ordered range scans, and remote synchronization. Evaluation still needs public structured datasets, one-million-row workloads, and independent researcher and hardware replications. WSL2 I/O counters remain incomplete; Windows sensitivity counters are workflow-level and do not isolate operation costs.

9 Conclusion

Revon demonstrates a content-addressed versioning path for structured key/value data through immutable objects, incremental Merkle updates, SQLite persistence, historical checkout, and adaptive differencing. Paired intervals and a three-seed sensitivity study support workload-specific differences between Revon-H and the forced-Merkle ablation; the long-history result remains inconclusive. The primary benchmark and follow-up runs do not establish performance across real schemas, public datasets, independent hardware, concurrent workloads, or ordered range queries. These measurements describe a reproducible prototype, not a production-system replacement.

Acknowledgment

The authors acknowledge the open-source Dolt project and the researchers whose dataset-versioning systems informed the experimental design.

References

- [1] Scott Chacon and Ben Straub. 2014. Git Internals - Git Objects. In Pro Git, 2nd edition. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [2] DoltHub. 2024. Dolt Storage Engine: Block Store. Dolt Documentation. <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [3] Tim Sehn. 2024. Dolt's Storage Engine. DoltHub Blog, February 29, 2024. <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- [4] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In Advances in Cryptology - CRYPTO '87, LNCS 293, pages 369-378. doi:10.1007/3-540-48184-2_32.
- [5] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. 1989. Making Data Structures Persistent. Journal of Computer and System Sciences, 38(1):86-124. doi:10.1016/0022-0000(89)90034-2.
- [6] Juan Benet. 2014. IPFS - Content Addressed, Versioned, P2P File System. arXiv:1407.3561.
- [7] Alex Auvolat and François Taïani. 2019. Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. 38th IEEE International Symposium on Reliable Distributed Systems, pages 221-230. doi:10.1109/SRDS47363.2019.00032.
- [8] Athicha Muthitacharoen, Benjie Chen, and David Mazières. 2001. A Low-Bandwidth Network File System. Proceedings of the 18th ACM Symposium on Operating Systems Principles, pages 174-187. doi:10.1145/502034.502052.
- [9] Anant Bhardwaj, Souvik Bhattacharjee, Amit Chavan, Amol Deshpande, Aaron J. Elmore, Samuel Madden, and Aditya G. Parameswaran. 2015. DataHub: Collaborative Data Science and Dataset Version Management at Scale. 7th Biennial Conference on Innovative Data Systems Research.
- [10] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya G. Parameswaran, and Amol Deshpande. 2016. Decibel: The Relational Dataset Branching System. Proceedings of the VLDB Endowment, 9(9):624-635. doi:10.14778/2947618.2947619.
- [11] Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. Proceedings of the VLDB Endowment, 10(10):1130-1141. doi:10.14778/3115404.3115417.
- [12] Souvik Bhattacharjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. Proceedings of the VLDB Endowment, 8(12):1346-1357. doi:10.14778/2824032.2824035.
- [13] Michael Armbrust et al. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. Proceedings of the VLDB Endowment, 13(12):3411-3424. doi:10.14778/3415478.3415560.
- [14] Attic Labs. n.d. Noms technical overview. GitHub. <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [15] Sheng Wang et al. 2018. ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications. Proceedings of the VLDB Endowment, 11(10):1137-1150. doi:10.14778/3231751.3231762.

Appendix A Reproducibility and Data Availability

The repository contains the benchmark harness, evidence audit, paired-ratio analysis, multi-seed robustness study, and manuscript validation tools. The primary corrected paper-profile bundle is evidence/paper-corrected-issues13-17-counterbalanced-final-20260923. The primary paired ratio intervals are in evidence/paper-corrected-issues13-17-counterbalanced-final-20260923/paired_ratio_uncertainty.csv. The Windows and WSL2 sensitivity bundles are evidence/paper-issues18-20-robustness-20260923 and evidence/paper-issues18-20-linux-wsl-20260923; the supplemental Windows telemetry bundle is evidence/paper-issues14-windows-telemetry-20260924. Their raw rows, manifests, audits, summaries, and seed-cluster intervals are retained. The trie-sensitivity bundle is evidence/trie-sensitivity-issues8-20260923. These artifacts validate evidence consistency; they do not independently reproduce timings. Project repository: <https://github.com/atharvasheersh/Revon>